Forty Grants, Four IDs: Executable Checks for Agent Delegation

Yuto Makino
The University of Electro-Communications, Japan
Swarm Alignment Hackathon | November 2026

Abstract

Agent swarms rely on delegated authority that must remain bounded as work moves through agents. We audit the public Agentic Principal Chain (APC) reference implementation at a pinned commit, using fictitious principals, valid signed envelopes, and its unchanged policy decision point. We vary a target feature or rule while controlling another blocker. A depth-three child under a signed depth-two ceiling is admitted when the irreversible-effects budget has headroom. The published depth-overflow variant is denied because that other budget is exhausted; the same denial occurs at legal depth. A second named variant intended to isolate ordered k-tuple detection is already blocked by a pairwise rule, although another upstream experiment does establish k-tuple behavior. In a three-way, depth-three delegation tree, 40 valid signed envelopes have only four identifiers. Revoking one leaf denies all 27 leaves at that depth. The same ID collision occurs among the planner, reviewer and deployer in APC's own DevOps case study. Five smaller matched grant graphs show APC off-target denials in every configuration and none in two independent frameworks. A minimal local patch passes the new depth and leaf-specificity checks and 47 relevant original tests. Supplementary tests expose signed-policy wiring and persistence-failure conditions. Our contribution is a reproducible, mechanism-level audit of this reference API and evaluation, not evidence of exploitation in a deployed service or of unsafe model behavior. The results provide reproducible regression cases for agent-delegation infrastructure.

1. Introduction

When an agent delegates a task, the next agent receives authority through a chain. A monitor must bind each action to the signed chain, its budget, and the history of prior actions. The Swarm Alignment Sprint explicitly asks how agent IDs survive copying or renaming and how an individual agent can be revoked [7]. APC is a recent, public architecture for this problem: it signs authorization envelopes and checks six conditions before an action is admitted [1, 2].

This study asks a narrower, testable question: do the supplied reference APIs and named evaluation cases enforce the specific delegation controls they appear to exercise? We examine a clean, pinned checkout of the unchanged APC implementation. This is a local implementation and benchmark audit. We do not attack a live service, infer behavior of an LLM, or dispute APC's conditional formal results.

We contribute two mechanism-specific counterfactual checks of published evaluation cases, a branch-local revocation counterexample using valid signed envelopes, matched graph controls in two independent Python frameworks, and a locally tested repair patch. We retain actual denial reasons and population effects: a correct blocked/admitted verdict alone cannot establish which control caused it or how many unrelated agents it disabled.

2. Related Work and Scope

APC defines a delegation budget that includes maximum depth and reports a blocked depth-overflow adaptive case [1]. Its authors and contributors already discuss a correct denial for the wrong reason: Issue #1 and an open paired-vector PR assert expected failed conditions [3, 4]. Other agent-authority work publishes executable conformance vectors with expected reasons and subresults [11]. Testing a condition's independent contribution is established practice [8]. We claim no new testing theory or first conformance suite. Here the specific

interventions are within APC conditions: depth versus irreversible effects inside budget, and pairwise versus ordered k-tuple rules inside composition closure.

Selective revocation of derived capabilities is also established. Biscuit assigns revocation identifiers to token blocks, while Tenuo specifies unique warrant IDs and parent-bound chains [5, 6]. We use those designs only as context, not as empirically tested baselines. APC's paper does not specify descendant-cascade revocation; we therefore test only whether revoking one uniquely intended leaf leaves unrelated leaves available.

3. Method

We pin the public bounded-agents repository to commit d31a1ea115a3e97ab972ac8d03f23ef36cf9b653, check that the checkout is clean, and record SHA-256 hashes for the relevant source files. The primary APC counterexamples call its unmodified Python reference API; the audit runner invokes the original adaptive cases and the full PolicyDecisionPoint. The repair is validated separately on a patched copy. The tested issuer holds the signing key; the failure is not a keyless forgery by an untrusted agent. The authorization probes use fictional resources, in-memory state and test keys, with no external model, service endpoint, real credential or recovered attack payload. Every issued envelope in the main APC counterexamples verifies under the test key.

For each named mechanism, we form a two-by-two matrix: vary the target property or rule while holding a competing blocker absent or present. A blocking effect is isolated only when changing the target changes the verdict with the competing blocker absent. A published comparison is informative only if the verdict changes along the intervention actually used. These are deterministic counterfactuals, not estimates of a population-level causal effect. We separately test valid signed delegation trees and exact-ID revocation.

Table 1. Audited contracts and the variables held fixed in each local test.

Contract	Fixed inputs	Intervention
Depth ceiling	Signed scope, session, action type/resource/cost, tracker	Envelope depth 2 or 3; pre-used irreversible effects 0 or 5
k-tuple attribution	Three actions and resources	Pairwise rule off/on; ordered rule off/on
Selective revocation	Signed three-way tree and PDP	Original versus unique-ID issuer; revoke one leaf

For population effects, we generate five small grant graphs (four balanced, one irregular) and issue each in APC, MandateKit and Behalf with unchanged native APIs. Three issuance-label settings are correlated controls, yielding 45 configured cases rather than 45 independent deployment observations. We check the selected leaf and every other grant before and after revocation, and copy one credential without reissuing it. A copied credential should share its grant ID; separately issued grants should retain distinct IDs even when their agent-label strings match. A display-alias change is modeled outside the signed SDK fields and does not establish a native rename capability. Three hand-selected artificial Behalf faults probe test sensitivity, not upstream vulnerabilities or representative mutation coverage.

4. Results

4.1 The depth case is blocked by another budget dimension

The upstream adaptive case makes a depth-three envelope with a signed maximum of two. It records two hops in BudgetState, then deliberately spends all five allowed irreversible effects and proposes an action costing one more. The returned reason is irreversible_effects: 6 > 5. The code comment explicitly calls this a proxy for demonstrating a ceiling [2]; it is not a hidden implementation detail.

Table 2. Depth overflow two-by-two ablation. Cells are full-PDP verdicts for the same signed scope and read action type/resource/cost; the actor is bound to each chain. The depth tracker is seeded to two hops in every cell; five pre-used irreversible effects exhaust that separate dimension.

Actual envelope depth	0 effects pre-used	5 effects pre-used
2, within cap	ADMIT	DENY: effects 6 > 5
3, over cap	ADMIT	DENY: effects 6 > 5

Removing only the unrelated pre-consumption makes the very same depth-three signed envelope admissible. At legal depth, the pre-consumption still causes denial. The reference narrow() signs a child without checking the depth ceiling, while the PDP checks BudgetState.current_delegation_depth rather than envelope.delegation_depth; ordinary proposed actions add zero delegation hops [2]. A separate run creates depth six under a cap of two, with a shared budget tracker that remains at zero, and admits all seven chain positions. The inference is limited to the supplied API path; a deployment could add an external issuer check.

4.2 A named k-tuple comparison is masked by a pairwise rule

The upstream attack_k_tuple_bypass() uses read, write, send_external. Its pairwise-only checker already contains a read-to-send_external prohibition, so both the pairwise-only and pairwise-plus-k-tuple variants deny. The published output itself reports a pairwise violation in the first case. With that pair removed, the same sequence is admitted without the k-tuple and denied with it. Thus the ordered rule works in the isolated probe, but adding it to the original configured checker has no verdict-level effect. A different upstream read, write, send_internal experiment does isolate k-tuple behavior; our finding concerns this one named variant [2].

Table 3. k-tuple two-by-two ablation. The published comparison is the lower row; both cells deny because the pairwise rule is already active.

Pairwise rule	k-tuple absent	k-tuple present
Absent	ADMIT	DENY: k-tuple
Present	DENY: pair	DENY: pair and k-tuple

A condition-level assertion would still mark both denials as composition_closure, just as depth and irreversible effects both fall under budget. Attribution therefore needs a check at the submechanism or budget-dimension level, with a control that removes competing blockers. This extends the scope of the already proposed condition-level conformance work [3, 4]; it does not replace it.

4.3 Branch identifiers collapse to delegation depth

AuthorizationEnvelope.narrow() sets a child's ID to its parent ID plus the new chain's depth. Children of the same parent therefore share an ID. Recursively, every node at the same depth under one root shares an ID, despite distinct principal chains and valid signatures. RevocationRegistry indexes only this ID [2].

Table 4. Matched three-way, depth-three trees. All 40 baseline actions are admitted. One leaf is revoked; APC has 26 off-target denials. Other frameworks are local positive controls under their stated revocation assumptions.

System / issuer	Grants	Unique IDs	Denied after one leaf revoke
Upstream narrow()	40	4	27
Bounded issuer demo	40	40	1
MandateKit 0.2.1	40	40	1
Behalf Python port	40	40	1

The same example shows an availability failure, not a privilege escalation: revoking a compromised leaf also denies 26 unrelated leaves. This is present even in APC's own Section 7 DevOps case-study fixture: signed planner, reviewer and deployer grants have the same ID, cs2-env:1:2; three scope-permitted, low-impact probe actions are admitted before revocation and all three are denied after revoking the planner's ID [2]. Other budgets and approval are held nonblocking to isolate revocation. A separate ancestor-revocation run allows existing descendants because the registry performs exact-ID lookup, but APC has no stated cascade contract. We do not count it as a formal counterexample.

4.4 Population effect across graph shapes

The same collision appears across five generated shapes and all three issuance-label settings. In each balanced APC tree of width b and depth d, the measured number of distinct IDs is d+1 and collateral leaf denials are b^d−1. The irregular tree has two unrelated denials. Under the configured local revocation states, MandateKit and Behalf deny only the selected leaf in every matched shape. All three systems treat a lossless credential copy as the same grant, so revoking the original also denies its copy. These observations concern grant identity, not physical identity of copied AI workers.

Table 5. Off-target denials after one leaf revocation, using the same-label issuance setting. Distinct-label and substituted-label settings give the same counts. These small shapes were selected for contract checks, not prevalence estimation.

Grant graph	Grants	APC	MandateKit	Behalf
Binary depth 1	3	1	0	0
Binary depth 2	7	3	0	0
Ternary depth 2	13	8	0	0
Binary depth 3	15	7	0	0
Irregular depth 3	11	2	0	0

The selected target's denial verdict and revocation reason remain correct even when unrelated grants are denied. In a separate sensitivity check with three artificial Behalf faults, target-verdict checks detect one of three, target-reason checks one of three, and a population contract three of three. The population check observes more grants; these counts are neither a published-suite comparison nor a general mutation score. APS reports related lifecycle and reason-specific conformance tests [11].

5. Discussion and Repair

Three boundaries need explicit binding. At issuance, the depth of a signed child should be checked against the signed ceiling and its grant ID made unique within the trust domain. At admission, the PDP should independently compare the presented chain depth with that ceiling and verify that runtime budget and composition state correspond to the signed envelope. At evaluation time, each negative case should name the precise mechanism expected to block it and include a positive control with other blockers disabled. These checks are complementary: an issuer check prevents creating bad grants; a PDP check contains issuer or wiring mistakes; a causal control makes the evaluation interpretable.

Our 17-insertion/3-deletion illustrative patch changes only APC core.py and pdp.py: narrow() rejects over-depth children and assigns UUIDv4 IDs, while the PDP checks signed envelope depth independently of the runtime tracker. On a clean pinned copy, direct signed depth-three grants are denied under cap two, legal depth is admitted, and a three-way depth-three tree has 40 IDs with only the selected leaf denied after revocation. Forty-seven original core, budget and PDP tests pass. This is a bounded regression, not a production patch: UUID uniqueness is probabilistic, old colliding grants need migration, the adaptive overflow fixture now raises at issuance, and cascade, persistence and composition binding remain open.

Supplementary checks illustrate the wiring risk. A signed child can add a read-confidential/external-send restriction while a session checker initialized from the root remains unaware; read then send are both admitted. Binding the checker to the child's restriction denies the second action. This is a trusted-configuration mismatch, not a malicious agent setting the checker. In a separate optional persistence test, a BudgetBackend.save() that explicitly returns False is ignored, and reloading the same session after a simulated restart admits two actions of cost 0.6 under a ceiling of 1.0. A successful-save control denies the second action. APC describes the persistence layer as outside its production-ready scope [1, 2]. Details are in Appendix B.

The study has one failure target, two independent controls, five small selected graph shapes, and deterministic in-memory scenarios. It cannot estimate prevalence in deployed agent systems, demonstrate an actual data leak, or show that a model discovered these paths. Existing proof results assume their stated conditions, including correct infrastructure state. The public reference API and two named evaluation cases do not, by themselves, establish the particular protections their labels suggest. The repair demonstrates only the tested depth and leaf-specificity properties.

Two independent implementations provide matched topology controls (Table 4). MandateKit 0.2.1 creates 40 signed grants with default UUIDs and, given complete fresh issuer-signed revocation lists, denies only the selected leaf; a separate test rejects two delegation edges under a root cap of one while accepting three legal cap/depth combinations. Its two documented Python suites pass 61 of 61 tests [9]. Behalf's Python port also creates 40 unique grant IDs with identical child labels and scope, admits all 40 baseline actions, then denies only the selected leaf; its three relevant upstream revocation tests and 82 local topology authorization observations pass [10]. Its full 114-test suite has one separate CLI quickstart failure and six optional skips. Behalf's topology test does not evaluate a maximum-depth policy. These systems differ in semantics and maturity; the controls establish leaf-specificity under their test assumptions, not an overall security ranking or distributed revocation freshness.

6. Conclusion

A signed envelope is useful only when each part of the runtime enforces the properties it carries. In the tested APC reference path, depth overflow passes when an unrelated budget has headroom, and one leaf's revocation can disable an entire depth layer. Two-by-two interventions expose the exact causes of reported denials and prevent a correct verdict from masking a missing control. The provided scripts make these findings repeatable and give concrete regression cases for agent-infrastructure builders.

References

[1] X. Muruaga. Bounded Agents: Delegation Security for Multi-Agent AI Systems. arXiv preprint, 2026. https://arxiv.org/abs/2608.15888

[2] X. Muruaga. bounded-agents reference implementation, pinned commit d31a1ea, 2026. https://github.com/xmuruaga/bounded-agents/tree/d31a1ea115a3e97ab972ac8d03f23ef36cf9b653

[3] avp9-nexus. Negative conformance vectors for C1-C6, GitHub Issue #1, 2026. https://github.com/xmuruaga/bounded-agents/issues/1

[4] avp9-nexus. Conformance: paired vectors with assertions on the expected failed condition, GitHub PR #4, 2026. https://github.com/xmuruaga/bounded-agents/pull/4

[5] Biscuit. Revocation guide and token-block revocation identifiers, official documentation. https://www.biscuitsec.org/docs/guides/revocation/

[6] Tenuo. Protocol specification v1, warrant identifiers and parent chains, official specification. https://tenuo.ai/spec/protocol-spec-v1

[7] Apart Research. Swarm Alignment Hackathon tracks, judging criteria, and submission guidelines, November 2026. https://apartresearch.com/sprints/swarm-alignment-2026

[8] K. J. Hayhurst, D. S. Veerhusen, J. J. Chilenski, and L. K. Rierson. A Practical Tutorial on Modified Condition/Decision Coverage. NASA/TM-2001-210876, 2001. https://ntrs.nasa.gov/citations/20010057789

[9] major-matters. MandateKit Python SDK 0.2.1, pinned commit fc6a3745, 2026. https://github.com/major-matters/mandatekit/tree/fc6a3745ef3eab5ee1e8842f275a5666cf9ffc4f

[10] novaai0401-ui. Behalf agent-authority Python port, pinned commit 556f91c, 2026. https://github.com/novaai0401-ui/agent-authority/tree/556f91cc7f356a74d625d798321ad998288f1ab0/python

[11] Agent Authority Conformance. Agent Passport System conformance suite, delegation and decision vectors, 2026. https://github.com/Agent-Authority-Conformance/aps-conformance-suite

Appendix A. Limitations and Dual-Use / Ethical Considerations

This audit tests a specific, public, research-oriented reference implementation under its documented in-memory API. It does not establish a vulnerability in a deployed service, a general failure of signed authorization, or a violation of APC's formal theorems. Depth overflow assumes the reference issuer is used without an added external depth gate. The composition mismatch assumes a trusted session checker is initialized before a child adds restrictions. The persistence case simulates a returned write failure and state reload, not a real process crash or lost funds. We did not inspect every source revision or every possible deployment wrapper.

The findings could help an operator test revocation or could help an adversary recognize weakly wired controls. To limit dual use, all examples use fictional principals and resources and make no network calls. We publish no live targets, credentials, exploit payloads, or instructions for compromising a running system. The source already exposes the relevant code and benchmark; our added contribution is bounded local verification and repair guidance. Any deployment should receive its own threat review and disclosure process before operational conclusions are drawn.

Appendix B. Reproduction and Additional Controls

Project code and saved results: https://github.com/makino-yuto/swarm-authority-conformance-2026

Environment: Python 3.11 or newer and clean pinned checkouts. The package README gives exact commands. src/run_apc_audit.py writes results/apc/audit.json; src/mechanism_conformance.py writes the two diagnostic matrices. src/graph_contract_campaign.py generates the five matched graph shapes and records native verdicts and artificial-fault sensitivity. src/apc_case_study_check.py imports the unchanged DevOps fixture. Each assertion must pass. The APC runner checks its source pin and hashes. No model calls or live transactions occur.

Depth ablation: cap=2; actual depths 2 and 3; tracker seeded at 2; action irreversible cost=1; pre-used effects 0 or 5. Depth three with no preload is admitted, while both depths with preload are denied specifically for effects $6 > 5$. An ordinary shared tracker stays at zero through repeated narrow() calls. The named k-tuple variant uses read, write, send_external; its pairwise rule alone blocks. Without that pair, toggling the k-tuple changes admit to deny. Another upstream send_internal case correctly isolates the ordered rule.

Branch test: one root, three children per node, depth three. All 40 envelopes are signed and initially admitted; 27 leaves share one ID and a single exact-ID revoke denies all 27. The bounded issuer gives 40 unique IDs, denies only the target leaf, and refuses depth four. The separate patch applies cleanly to the pinned source, adds issuance and PDP depth checks plus UUIDv4 IDs, and passes 47 relevant original tests. Neither repair establishes distributed uniqueness, migration or cascade policy.

Additional policy-state tests: a child restriction is signed, but a checker initialized only from the root permits read then external send; a checker bound to the child denies the send. A BudgetBackend returning False on its first save admits two 0.6-cost actions under a 1.0 cap after same-session state reload; the successful-save control denies the second. These require the stated integration conditions. Three hand-selected artificial faults test the sensitivity of target-only versus population-wide assertions; they are not defects discovered in Behalf. Behalf's full-suite CLI failure is retained in its result JSON; only the relevant core tests are presented as passing.

LLM Usage Statement

The sole author set the research objective and constraints. OpenAI Codex conducted most source inspection, generated the audit scripts, ran local probes, analyzed results, and drafted this report under that direction. The author remains responsible for the final submission. Every reported numerical result is tied to a saved program run; no LLM-generated empirical value was inserted without corresponding output. An independent human rerun has not been claimed.